Aikido-proof build instructions

For: the coding agents and the 3 developers who build our hackathon app. **Goal:** an app that passes the Aikido Security scan (including the AI Code Audit for login, permissions, IDOR and business logic) because it is **secure by construction**, not because problems are hidden. **Version:** 18 October 2026.

> Zero findings can never be guaranteed. Aikido-proof means: secure by construction, tested, and documenting the one or two choices we made on purpose. It does **not** mean hiding code from the scanner.

The document at a glance

Part	Name	What's in it	Who uses it	When
Plan	Step-by-step plan	What to do, who does it, how long, and the exact prompt to give the agent	All 3 developers	Read first, follow during the hackathon
Section 0	Background	What Aikido checks and how it scores	Developers	Read once, before the hackathon
Section 1	The 15 rules	The most important rules on one page	Developers	Read once, before the hackathon
Section 2	Detailed rules	Every rule with an ID (R-AUTH-01, R-BIZ-05, …): what to do, what not, how to check	Agents (and developers when in doubt)	Looked up per feature, during step 3
Section 3	Agent contract	The rules every agent must always follow, ready to paste	Agents	Pasted into `CLAUDE.md` in step 0, active the whole time
Section 4	Final checklist	Commands, greps and attacks to run before submitting	1 developer + agent	Step 5, last 30 minutes
Section 5	Ready-made code	The security core: login, sessions, permissions, rate limits, CSRF, headers, plus an example route	Developer 1 + agent	Step 2, during the brainstorm (or beforehand as a template)
Section 6	README template	The `## Security` section the jury and the auditor read	1 developer	Step 5
Section 7	Common agent mistakes	What agents typically get wrong, and the fix	Developers reviewing agent code	During step 3 and step 5
Links	Aikido documentation	Official Aikido pages, for when you're unsure what Aikido checks	Agents and developers	When in doubt

Step-by-step plan

Total security work: about 50 minutes, spread over the hackathon. Everything else is normal feature work, done the secure way from the start.

Step 0: Prepare the repo (5 min, developer 1, before or right after the kick-off)

1. Create the GitHub repo and put this file in the root as `AIKIDO-INSTRUCTIONS.md`.
2. Create `CLAUDE.md` in the root and paste **section 3** into it. If someone uses Codex or Cursor, copy it to `AGENTS.md` too.
3. Ask the organizers: how do Aikido findings count for the jury, and must the repo be public?

Step 1: Decide the security model (10 min, all 3 together, right after choosing the solution; step 2 is already running meanwhile)

First decide: does the app need a login? Aikido doesn't require one. It checks that whatever you build is secure.

Login needed?	When
No login	Everything is public and the same for everyone (a calculator, a map, a public dashboard). There is no personal data, and users can't change anything that others see. No login is then *safer*: no passwords, sessions or permissions, so fewer things Aikido can find.
Login (with guest mode for the demo)	Any of these: users have their own data (profile, documents, bookings, credits); users change things others see (votes, reviews, approvals); there are roles (admin vs. member) or separate groups (companies, clubs); there's private or sensitive data (medical, financial, personal). Without a login, the AI Code Audit reports this as *broken access control*. Give the jury a read-only **guest mode** (R-AUTH-04), so they don't need a password.

When in doubt, choose **login with guest mode**: it's safe, and the jury still gets in with one click.

Then write down in the README, in a few lines:

- **login: yes or no**, and why (one line);
- the **roles** (for example member, coach, admin, guest);
- the **tenants**, meaning the groups whose data must stay separate (companies, clubs, schools);
- the **objects** and who may see or change them;
- every **money, credit, vote, booking or approval flow**.

This list decides "who may do what". All three developers and their agents must use the same answer.

Optional prompt:

> Read our case and solution in README.md. Following rule 1 of section 3 in AIKIDO-INSTRUCTIONS.md, write a short "Security model" section in README.md: first whether the app needs a login (using the table in step 1), then roles, tenants, objects with who may read and change them, and every money, credit, vote, booking or approval flow. Ask me about anything unclear instead of guessing.

Step 2: Build the security core (developer 1 with an agent, in parallel with the brainstorm)

The security core doesn't depend on the idea, so **don't wait for the solution**:

- **Best, if the hackathon rules allow pre-written code:** a starter template made before the hackathon. At the event you only adapt the roles (about 5 min).

- **Otherwise, during the brainstorm:** at the start of the brainstorm, developer 1 gives the agent the prompt below and then joins the brainstorm. The agent works on its own for about 20 minutes, and developer 1 checks in now and then.
- **After step 1:** adapt `users.ts` and the roles to the security model (about 5 min), using the second prompt below.

The agent builds the foundation from **section 5** (ready-made code: the agent copies it, it doesn't invent its own). Developers 2 and 3 don't write routes until the core is pushed.

Prompt for developer 1:

> Follow the build order at the top of section 5 in AIKIDO-INSTRUCTIONS.md. Scaffold the Next.js app if it doesn't exist yet. Copy every file from section 5 exactly (src/lib, proxy.ts, the login and logout routes, scripts/create-users.mjs), adapt users.ts and the roles to the security model in README.md, merge the headers into next.config.ts, and extend .gitignore and .env.example. Create demo users with create-users.mjs and put the output in .env.local (never commit it). Then run lint, typecheck, build and the login tests, and commit.

Prompt for developer 1 after step 1 (adapt the core to the idea):

> Read the security model in README.md. Adapt src/lib/users.ts, the roles in auth.ts and access.ts, and the demo users to it. Keep every security behavior from section 5 unchanged. Run the tests again and commit.

If step 1 decided "no login": remove the login part after step 1 with this prompt (about 5 min):

> Our security model in README.md says the app has no login. Remove the login, logout and signup routes, proxy.ts, auth.ts, users.ts, password.ts, access.ts and scripts/create-users.mjs, plus SESSION_SECRET and AUTH_USERS from .env.example. Keep http-error.ts, store.ts, limit.ts, validate.ts, csrf.ts and the security headers. Because nobody logs in, every route must be safe for anyone: rate-limit every route per IP (limit.ts clientIp, on Vercel) plus a global cap per route, especially AI and other paid API calls; validate every input; never store or show personal data. Run lint, typecheck and build, and commit.

Even without a login, these stay required:

- rate limits on every route, and especially on AI and other paid APIs (otherwise anyone can run up your bill);
- input validation;
- no XSS;
- no secrets in the code;
- security headers;
- the final checklist.

When it works: **push to `main` and tell the others.** From now on, nobody writes their own login, session or permission check. Everyone uses these files.

Step 3: Build the features (the main part, all 3 in parallel, each with their own agent)

- **Split by feature**, not by layer. Each developer owns their routes from start to finish.

- **Every route follows the same order:** Origin check → `requireUser()` → rate limit → schema → access check → one atomic write → minimal response. The example route in section 5 shows it.
- **Before a feature, point the agent to the right rules:**

Feature type	Agent reads first (section 2)
Anything with users, roles, teams	R-AUTHZ-01 to R-AUTHZ-05
Money, credits, votes, bookings, approvals	R-BIZ-01 to R-BIZ-07
Sign-up, guest mode, demo accounts	R-AUTH-04, R-AUTH-07, R-AUTH-08
AI / chat / RAG / tools	R-AI-01 to R-AI-05
File upload, download, share links	R-FILE-01 to R-FILE-03
Forms and any user input	R-VAL-01 to R-VAL-06

Prompt template for each feature:

> Build [feature]. Follow AIKIDO-INSTRUCTIONS.md: read [the rules from the table] first, and make every route follow the example route in section 5. Use the existing src/lib files; don't write your own auth or permission checks. Add the attacks for this feature to scripts/attack-tests.mjs (no login, a user of another tenant, 20 parallel requests where relevant), run them, and commit.

Team rules during step 3:

- Changes to `src/lib/auth.ts`, `store.ts`, `limit.ts`, `access.ts` or `csrf.ts` are announced to the team; only developer 1 changes them.
- Never commit `.env.local`, passwords or real keys. Real keys go in Vercel env vars; share them via a password manager, never in chat or the repo.
- Pull often, and make small commits.
- When you review agent code, check it against **section 7** (common agent mistakes).

Step 4: Keep the attack tests running (during step 3, a few minutes per feature)

After each feature, run `scripts/attack-tests.mjs` against `npm run build && npm start`. A red test is fixed right away, not "at the end".

Step 5: Final check (last 30 minutes, split over 3 people)

- **Developer 1, the checklist (section 4)**, with this prompt:

  > Run the full pre-submit checklist in section 4 of AIKIDO-INSTRUCTIONS.md: the repo checks, npm audit, lint, typecheck, build, all greps, the unit tests and scripts/attack-tests.mjs against npm start. List every failure and fix the real problems. For anything we keep on purpose, write a one-line trade-off for the README.

- **Developer 2, the README security section (section 6):** for each protection, the file it lives in and its test, plus the trade-offs from developer 1.
- **Developer 3, GitHub (R-IAC-04):** check the git history for secrets. Make the repo public, if allowed (needed for branch protection on a free account). Add the small CI workflow, then turn on branch protection for `main`. From then on, every fix goes through a pull request with one review.

Step 6: After submitting (optional)

If you have Aikido credits: connect the repo, run the scans and the AI Code Audit, and ignore findings you accept *one by one* with a reason (never whole folders).

Other stacks

The code in section 5 is for Next.js + TypeScript on Vercel (optionally Upstash Redis). The rules in sections 1–4, 6 and 7 apply to any web app. If the case needs another stack, build the same pieces with the same behavior:

Piece (section 5)	Next.js	FastAPI (Python)	Express (Node)
Session cookie, signed, revocable	`auth.ts`	`itsdangerous` signer or a random token in Redis, `HttpOnly` / `Secure` / `SameSite` cookie	same as `auth.ts` with `cookie-parser`
Passwords	`password.ts` (scrypt)	`hashlib.scrypt` or `argon2-cffi`, `hmac.compare_digest`	`node:crypto` scrypt
Session check in every route	`requireUser()`	a `Depends(require_user)` dependency on every router	middleware on every router, plus a check in each handler
Schema validation	zod `.strict()`	Pydantic with `extra="forbid"`	zod
Atomic changes	store `transfer` / Lua	Redis Lua or a DB transaction	Redis Lua or a DB transaction
Login limiter (reserve, refund)	`limit.ts`	the same logic with Redis `INCR` / `DECR`	`limit.ts`
CSRF (`Origin` vs `Host`)	`csrf.ts`	a small middleware for POST/PUT/PATCH/DELETE	`csrf.ts` as middleware
Security headers	`next.config.ts`	a middleware that sets the same headers	`helmet` plus the same CSP

Aikido's own documentation (links)

When an agent or developer is unsure what Aikido checks or how something works, look it up here first. These pages are public; an agent can read them with a web fetch. Start with the index, which lists every help page:

- **Index of all help pages (best starting point for agents):** https://help.aikido.dev/llms.txt
- **Help center:** https://help.aikido.dev
- **AI Code Audit (login, permissions, IDOR, business logic):**
- https://www.aikido.dev/code/ai-code-audit
- https://help.aikido.dev/ai-code-analysis/ai-code-audit-overview
- **SAST, which languages and what it focuses on:** https://help.aikido.dev/code-scanning/scanning-practices/sast-by-aikido-supported-languages-and-security-focus
- **Dependency scanning, which lockfiles are read:** https://help.aikido.dev/code-scanning/scanning-practices/support-for-dependency-scanning-by-language
- **Secrets detection:** https://help.aikido.dev/code-scanning/scanning-practices/live-secret-detection
- **License issues:** https://help.aikido.dev/getting-started/core-functionalities/display-license-issues-in-feed

- **GitHub repository settings (access control checks):** https://help.aikido.dev/code-scanning/scanning-practices/enhancing-security-with-access-control-checks
- **The `.aikido` file (exclude paths, ignore CVEs):** https://help.aikido.dev/code-scanning/scanning-practices/ignore-via-code-with-aikido-files
- **Ignoring a single finding in the UI:** https://help.aikido.dev/getting-started/core-functionalities/ignore-issues-to-remove-issues-from-main-feed
- **AutoFix:** https://help.aikido.dev/autofix-and-remediation/overview-aikido-autofix
- **IDOR detection (pentest):** https://help.aikido.dev/pentests/coverage-and-findings/what-issues-can-aikido-pentest-find/understanding-and-detecting-idor-vulnerabilities
- **Testing AI features (pentest):** https://help.aikido.dev/pentests/configure-a-pentest/testing-ai-capabilities
- **Zen runtime firewall for Next.js** (optional extra; it doesn't replace the checks in the code): https://help.aikido.dev/zen-firewall/zen-installation-instructions/zen-firewall-for-nodejs-javascript-and-typescript/nextjs

Only in the Aikido app (login needed, so agents can't open them):

- the full SAST rule list per language: https://app.aikido.dev/repositories/sast
- the IaC, mobile and GitHub access-control rule lists.

The rules from those lists that matter for our stack are already worked into section 2 (look for the rule IDs R-VAL, R-SEC, R-AUTH-09, R-DEP and R-IAC).

If the docs and this file disagree: the Aikido docs win for *what Aikido checks*; this file wins for *how we build it*. Tell the team, so we can update this file.

Section 0: Background, what Aikido checks

- **The AI Code Audit reads source code only.** It doesn't crawl a running app or replay traffic, and needs nothing more than a connected repo. Pricing is in credits, fixed before the audit starts.
- It follows data flow, ownership checks and permission boundaries across files.
- It looks for IDOR, broken access control, business-logic bypasses, race conditions, broken auth checks, and chains of several low-severity issues.
- Sources: Aikido Code Audit, Aikido changelog, OWASP blog.
- **What this means for us:** the auditor has to be able to *see* each check. Put the check in the code next to the data access (in the route handler, server action or data-access function), not only in `proxy.ts`.
- **Aikido Pentest is a separate product** that tests a running app. For IDOR it logs in as a test user and swaps IDs in requests. A clean Code Audit does not mean a clean pentest, or the other way round. (Aikido docs)
- **Other scanners:** SAST, secrets, dependencies, lockfiles and licenses (SCA), code quality, IaC, containers and cloud. There is also Zen, a runtime firewall, which is extra protection and does **not** replace checks in the code.
- **The `.aikido` file** in the repo root is read on every scan. (Aikido docs) ```yaml exclude: paths:
  - data/fixtures # matched as a plain substring of the file path, no wildcards ignore: cves: CVE-2025-22869: reason: Only used in a build script, never reachable at runtime ```
- An excluded path is skipped by the secrets, SAST, lockfile, code-quality and Deep Review scans. Because it's a substring match, `test` would also exclude `src/latest/...`.
- Prefer ignoring **one specific finding in the Aikido UI** over excluding whole folders. (Aikido: ignore issues)
- **Unknown, so don't rely on it:**
- whether `.aikido` `exclude` also applies to the AI Code Audit;
- whether Aikido honors inline comments like `nosemgrep` or `gitleaks:allow`;
- whether the auditor reads the README. Write the `## Security` section anyway (R-DOC-01): it costs 5 minutes and helps the jury.
- **The jury's scoring formula is unknown.** We don't know how Aikido findings count, or whether ignored findings cost points. **Ask the organizers at the start.**

Section 1: The 15 most important rules

1. **Every route handler, server action and data-access function checks the session itself.** `proxy.ts` and hidden UI are not authorization.
2. **The user's identity always comes from the session**, never from the body, the query string, a header or LLM output.
3. **Every lookup, change, download, export and search checks owner or tenant.** Deny by default, and return 404 on a mismatch.
4. **Votes, credits and one-time actions are atomic** (Redis `SET NX`, a transaction, a unique constraint). No "check, then write".
5. **Prices, roles, flags, numbers and workflow steps are decided and checked on the server.**
6. **Every input goes through a strict schema** (`.strict()`, min/max) with a body size limit. Copy only allowed fields; never spread the raw body.
7. **Signed or opaque sessions** in `HttpOnly`, `Secure`, `SameSite` cookies with an expiry, revoked on the server at logout. **Fail closed** if the session store is down.
8. **Rate-limit login, sign-up, guest, upload and LLM routes with Upstash**, never in-memory. Make the limiter fail closed.
9. **Secrets only in server env vars** (`import "server-only"`), never in `NEXT_PUBLIC_`, git, prompts or logs. A leaked key must be rotated.
10. **No SQL or NoSQL string concatenation, `eval`, `exec`, or paths built from user input.**
11. **No server-side `fetch` of a URL from a user or a model**: use fixed endpoints only.
12. **Render user and LLM content as text.** No `dangerouslySetInnerHTML`, no raw HTML in Markdown, no `javascript:` links.
13. **Route handlers that change data check `Origin`** (CSRF), with no wildcard CORS. Set security headers, and return generic errors with minimal data.
14. **RAG retrieves only documents the user may see, and every LLM tool checks permissions itself.** Cap tokens and the number of calls.
15. **Commit the lockfile, `npm audit` it, and run the negative tests** (no cookie, user A requests B's data, a forged role, a replayed request, concurrent requests) before submitting.

Section 2: Detailed rules (look up per feature)

2.1 Scan and triage

- **R-SCAN-1: Scan the commit you're submitting.** Compare the commit in the Aikido dashboard with `git rev-parse HEAD`. Look at the Code Audit separately from SAST, secrets and SCA.
- **R-SCAN-2: Fix first. Ignore only one specific, tested finding, with a reason.** Never use broad path excludes or blanket ignore comments just to raise the score. An accepted risk is not a false positive: a false positive is one that can't actually be exploited.

2.2 Authentication

- **R-AUTH-01: Signed or opaque sessions in a secure cookie.** (section 5 `auth.ts`)
- Do: an HMAC-signed cookie with an expiry (section 5), *or* a random 32-byte token whose hash is stored in Redis.
  - Cookie flags: `HttpOnly`, `Secure` in production, `SameSite=Lax` / `Strict`, `Path=/`.
  - In production, prefix the name with `__Host-` (requires HTTPS, `Secure`, `Path=/` and no `Domain`).
  - Revoke at logout on the server. Changing a password ends all sessions.
  - Pattern (section 5 `auth.ts`): the cookie holds a random **session ID** (`sid`) and a **password version** (`pv`, the first characters of the password's salt). Logout puts the `sid` on a revoked list in Redis until the cookie would expire anyway. A new password gives a new salt, so all old cookies stop matching.
  - `SESSION_SECRET` is at least 32 characters. Without it, login refuses to work (503) instead of signing with an empty key.
- Don't: an unsigned JSON or base64 cookie, tokens in `localStorage`, or logout that only deletes the cookie.
- Check: change one character in the cookie and expect 401. Reuse an old cookie after logout and expect 401.
- Severity: critical.
- **R-AUTH-02: Hash passwords with scrypt or Argon2id, and compare in constant time.** (section 5 `password.ts`)
- **No account enumeration through timing:** when the user doesn't exist, still hash against a dummy salt and hash, so the response takes just as long.
- Cap the password length (for example 8–200 characters), so a huge password can't make the server spend forever hashing.
- **Demo accounts without passwords in the repo:** a script (`scripts/create-users.mjs`) generates `id:salt:hash` lines for the `AUTH_USERS` env var. The real passwords live only in the team's password manager.
- High.
- **R-AUTH-03: Rate-limit login: count first, per account *and* per IP, never a permanent lockout.**
- **Reserve the attempt *before* checking the password, and refund it after a correct one** (atomic `INCR` first, refuse when over the limit, `DECR` on success). Don't "peek, check the password, then count the failure": parallel guesses all pass the peek. Don't count successes either. Otherwise a few wrong guesses by someone else block the real user's *correct* password.
- **Three counters:** per account+IP (5 per 15 min, strict), per IP (50 per 15 min), and **per account from any IP** (100 per 15 min). Without the last one, rotating IPs gives unlimited guesses on one account. It's set high, so it slows attackers down rather than locking the real user out. Write down the trade-off: someone who sends 100 wrong guesses can make a colleague wait up to 15 minutes.

- **Client IP only from a header the platform guarantees.** On Vercel, `x-forwarded-for` is overwritten by Vercel, so clients can't fake it (Vercel docs). Use it only when `process.env.VERCEL` is set (Vercel sets `VERCEL=1` at runtime when system env vars are enabled, the default). Anywhere else, treat the IP as unknown and rely on the per-account counter. **Never** let all unknown IPs share one bucket called `"unknown"`, or 50 bad logins lock everyone out. (Code: `clientIp()` in section 5.)
- Use the same error message and timing whether the account exists or not.
- Check: 20 parallel wrong logins with `VERCEL=1` and one fixed IP → at most 5 get evaluated. Without `VERCEL`, `x-forwarded-for` rotation makes no difference. After 10 wrong guesses, the correct password still works. A different account can still log in. Test the IP limits locally by starting with `VERCEL=1 npm start`.
- Medium–high.
- **R-AUTH-04: Guest and demo accounts can do little and see only their own data.**
- The guest is **read-only**: it can ask questions but can't vote, so it can't change anything other users see. Creating guest sessions is rate-limited per IP.
- Make guest mode switchable with an env var (for example `GUEST_ACCESS=off`). Switching it off also ends existing guest sessions, because the guest's password version disappears.
- Don't: `admin:admin`, a public admin link, a role switch in the UI, or `catch { return demoUser }`.
- High.
- **R-AUTH-06: Check on *every* request that the user still exists and is still active.** (section 5 `findActiveUser()`)
- Users who left the company, or a deleted account, are refused even while their cookie is still valid.
- Don't: trust whatever is in the cookie for 8 hours.
- High.
- **R-AUTH-07: Make sign-up atomic, rate-limited and kept apart from built-in IDs.**
- Create the account with `HSETNX` (or a unique constraint), so two sign-ups for the same email can't both succeed.
- Allow 5 sign-ups per IP per hour. Normalize the email (trim, lowercase, at most 254 characters).
- Give self-made accounts their own prefix (`acct:<email>`), so nobody can sign up as `admin`, `guest` or an existing demo ID.
- Write down the known trade-off: one person can create several accounts.
- **Anything a new account gets for free is multiplied by fake sign-ups.** Example: EUR 50 per sign-up plus trading or gifting lets an attacker move EUR 50 per fake account to themselves, and stuff votes and leaderboards. So:
  - no free credit that can leave the account (it can only be *spent*, not traded, gifted or sold);
  - votes, rankings and rewards only for seeded or verified accounts, or list it as an accepted trade-off;
  - leaderboards ignore trades between an account and accounts created after it, or anything below a minimum.
- **Sign-up reveals whether an email exists.** Even a generic "Could not create this account" leaks it, because success still looks different. The only real fix is email verification that shows "check your inbox" in both cases. Without an email service (normal at a hackathon): keep the generic message, rate-limit sign-up, and list it as a trade-off.
- When the case asks for gifts or transfers between users: keep **two balances**, free (spend-only) and earned (transferable). Pick recipients by a handle or gift code, never by looking up an email.
- High.
- **R-AUTH-05: Fail closed.** If Redis or the session check fails, return 503, never guest or admin access.
- Don't: `try { verify() } catch { return true }`.
- Critical.
- **R-AUTH-08: Demo logins for the jury without passwords in the repo.**

- Commit only hashes (`AUTH_USERS` in the Vercel env, generated by `scripts/create-users.mjs`). Put the demo username and password in the **submission form or pitch slide**, not in the repo. Use "Continue as guest" so the jury doesn't need a password at all.
- For local testing, put test passwords in `.env.local` or an untracked `TEST-ACCOUNTS.local.md` (add `*.local.md` to `.gitignore`).
- If a demo password must be in the README, the account can't do anything a guest can't, and it's listed as an accepted trade-off.
- **R-AUTH-09: Tokens and crypto that scanners check literally.**
- **JWT:** if you use JWTs at all, only `jwt.verify(token, key, { algorithms: ["HS256"] })`. Never `jwt.decode()` to trust claims, never `alg: "none"`. (The section 5 session isn't a JWT, so this doesn't apply there.)
- **Comparing secrets:** passwords, tokens, signatures and API keys are compared with `timingSafeEqual` (Python: `hmac.compare_digest`), never with `===`. Aikido flags `===` on passwords as a timing attack (High).
- **TLS stays on:** never `NODE_TLS_REJECT_UNAUTHORIZED=0`, `rejectUnauthorized: false` or `verify=False`. Database connections use TLS (`?sslmode=require`, Sequelize `dialectOptions.ssl`). Websockets use `wss://`, never `ws://`.
- **No third-party scripts from risky CDNs:** never `polyfill.io` or `bootcss`-type domains (Aikido marks these as critical "malicious domain"). Prefer self-hosted packages from npm; if you must use a CDN, pin the version and add `integrity=` (SRI).
- **R-OPS-01: The shared store fails closed in production.**
- Locally, no Upstash means a memory store, which is fine.
- In production (`NODE_ENV=production`) without Upstash env vars, every store call returns **503**, unless `ALLOW_MEMORY_STORE=true` is set on purpose for a single-instance demo (and listed as a trade-off). Otherwise, on Vercel each instance has its own sessions, votes, limits and balances, which allows double spending and makes logout ineffective.
- Check this *lazily* (on first use), not when the module loads, or `next build` crashes.

2.3 Authorization and IDOR

- **R-AUTHZ-01: Check in every entry point.** (Next.js data security)
- Every route handler, server action and server-side page that loads data calls `requireUser()`, then checks permission.
- Next.js itself says: "*treat Server Actions as reachable via direct POST requests and verify authentication and authorization inside each one.*"
- A check at page level does **not** protect the server actions on that page.
- The proxy is the *first* gate, and it denies by default. It has an explicit list of public paths (for example `/login`, `/api/auth/login`, `/api/auth/signup`, public share links). Everything else needs a session: `/api/*` gets a 401 JSON response, and pages redirect to `/login`. Route handlers and pages check again, close to the data. (section 5 `proxy.ts`)
- Critical.
- **R-AUTHZ-02: Identity comes only from the session.**
- Don't: `const { userId, isAdmin } = await req.json()`. IDs in the request may only say *which object* is meant, never *who is asking*.
- Check: `grep -rnE "userId|ownerId|tenantId|isAdmin|role" src/app` and look at every value that comes from the body, `searchParams` or `params`.
- Critical.
- **R-AUTHZ-03: Look up every object by ID *and* owner or tenant.**
- Do: `where(and(eq(doc.id, id), eq(doc.ownerId, user.id)))`, or an access helper (section 5 `access.ts`). For changes, repeat the condition *in* the atomic write itself, not only beforehand.

- Don't: look up by `id` alone and count on UUIDs being impossible to guess.
- Check: as A, send GET, PATCH, DELETE, download and search requests with B's ID. Expect 404, and B's data unchanged.
- Critical.
- **R-AUTHZ-04: Field permissions and mass assignment.**
- Do: a zod schema with `.strict()` for writable fields, and roles from a server-side store.
- Don't: `db.user.update({ data: body })`, or accept `role`, `approved`, `balance` or `ownerId` from the client.
- Check: POST extra fields like `isAdmin:true`, `price:0`, `ownerId:B`. Expect a 400 or no effect.
- High.
- **R-AUTHZ-05: No leaks through caching or full objects.** (Next.js recommends minimal DTOs)
- Do: `Cache-Control: private, no-store` on private data; cache keys that include the user or tenant; return only the fields you need, from both server actions and props to client components.
- Don't: return a whole database row, or cache private data under a shared key.
- Next 16: never put `"use cache"` on functions that return per-user data. Reading `cookies()` keeps a route dynamic; check that private pages show `ƒ (Dynamic)` in the `next build` output.
- High.

2.4 Business logic

- **R-BIZ-01: Make one-time actions atomic and idempotent.**, and in the Code Audit's scope (race conditions)
- Do: `const first = await redis.set(\`vote:${claimId}:${user.id}\`, 1, { nx: true }); if (!first) return 409;`, or a transaction with a unique constraint.
- Don't: `if (!(await redis.get(k))) { await redis.incr(count); await redis.set(k, 1) }`, because two parallel requests both get through.
- Check: send 20 identical requests at the same time. Exactly 1 should succeed.
- High.
- **R-BIZ-02: Check numbers, prices and steps on the server.**
- Do: `z.number().int().min(1).max(10)`. Look up prices on the server, in integer cents. Allow only transitions from a fixed list of valid previous states.
- Check: send -1, 0, 1e300, 1.5, a changed price, and the last step without the earlier steps.
- High.
- **R-BIZ-03: Resist replay.** Tokens and one-time links expire and stay tied to the user and the action. High.
- **R-BIZ-04: An action only counts where the object actually applies, and where the user has access.**
- Example: a vote or rating on an item only counts in a context (team, region, client, project) where that item is actually shown, *and* only in a context the user belongs to. Otherwise someone could vote to hide or boost an item in a context they don't belong to, and change what other people see.
- Store the vote under the key `userId:objectId`, so each user has at most one vote per object (a new vote replaces the old one).
- Actions that affect what *others* see (votes, ratings, shared edits) need: a real account (not a guest), a rate limit per user (for example 60 per minute), and this context check.
- High.
- **R-BIZ-05: Changes to several records happen in *one* atomic step.**
- Examples: a purchase changes 4 balances; an approval changes the shift owner *and* the points.
- Do one of these:
  - one Redis **Lua script** (`EVAL`) that checks and changes all keys (section 5, `transfer.lua`);
  - keep everything one tenant changes in **one document** and do compare-and-set on its version;

    - one database transaction with the checks inside it.
- Don't: a lock with a TTL (`SET lock NX PX 5000`) around several separate calls. If the calls are slow, the lock expires halfway and a second writer gets in (a double spend).
- Check: 20 parallel buys or transfers → totals still add up (nothing created or lost).
- High.
- **R-BIZ-06: Re-check every business rule at *every* step that commits it, especially the last one.**
- At each state change, check again: the rule itself (no double booking, limits, balance), whether the date or time is still valid (can't approve a past shift), and whether the actor and the other party are still active members of the tenant.
- Check: create two pending requests that are each fine alone but break a rule together, then approve both. The second must fail.
- High.
- **R-BIZ-07: Public pages show as little as possible.**
- Never derive a public name from the email. Use a chosen display name or a pseudonym.
- A public leaderboard shows only the name and score: no IDs, emails or exact timestamps.
- Medium.

2.5 Input and injection

- **R-VAL-01: Strict schemas on everything.**
- Applies to the body, `params`, `searchParams`, headers you use, webhooks, LLM tool arguments and file metadata.
- Limit the body size *while reading it*, not afterwards.
- Broken JSON gives 400, too large gives 413.
- Use `.strict()` so unknown fields are rejected.
- Don't: `as MyType`. (Code in section 5.)
- High.
- **R-VAL-02: No injection.**
- Do: parameterized queries or ORM calls. Use a fixed filter shape for NoSQL.
- Don't:
    - concatenate strings into SQL;
    - pass client JSON straight into a `where`;
    - use `$where` or `$ne` from the client;
    - use `eval`, `new Function` or `exec(userText)`;
    - use `shell: true`;
    - in Python: `pickle.loads` or `yaml.load` on untrusted data, or `subprocess(..., shell=True)`.
- These Aikido SAST rules also fire on JS/TS:
    - `setTimeout`/`setInterval` with a **string** as first argument (they count as `eval`);
    - `child_process` (`exec`, `execSync`, `spawn` with `shell`) with any variable in the command;
    - `js-yaml` loading untrusted YAML with a schema that can build JS objects;
    - an XML parser with external entities on (`noent: true`).
    - SQL built with a template string or `+` (Sequelize, `sqlite3`, `pg`, raw Prisma `$queryRawUnsafe`) is flagged even if the input is "probably safe". Always use parameters.
- Python (FastAPI scripts), flagged by name:
    - `eval`/`exec`, `pickle`/`marshal`, `yaml.load` (use `yaml.safe_load`), `subprocess` with `shell=True`, `torch.load(weights_only=False)`;

    - `xml.etree` / `minidom` / `sax` / `lxml` on untrusted XML (use `defusedxml`);
    - `tempfile.mktemp` (use `mkstemp`);
    - `requests(..., verify=False)`;
    - Flask `debug=True`, Jinja2 without autoescape;
    - `assert` used for a security check (it's skipped in optimized mode).
- Critical.
- **R-VAL-03: No path traversal or prototype pollution.**
- Open documents by a **known ID from your own index** (`docs.find(d => d.id === id)`), never by a file path or name from the request. Then check access to that document.
- Do: generate file names on the server, and keep only allowed keys when merging objects.
- Aikido flags any `path.join` / `path.resolve` with a request value that later reaches `fs.readFile` / `createReadStream` (and Express `res.sendFile`, Supabase Storage paths, zip extraction). Read files only from a fixed folder by a server-generated ID.
- Check: send `../`, `%2e%2e%2f` and `__proto__`.
- High.
- **R-VAL-04: No SSRF.**
- Do: fixed URLs in the code. If the case really needs fetching a user URL, use an allowlist of exact HTTPS hosts, turn off redirects (`redirect: "manual"`), and set a timeout.
- This includes headless browsers: a user URL passed to Puppeteer/Playwright `page.goto()` is flagged as critical SSRF.
- Don't: rely on `startsWith("https://")` or on blocking `localhost`. Blocking private IP ranges alone can be bypassed with DNS rebinding or redirects.
- Critical.
- **R-VAL-05: No XSS.**
- Render as normal JSX. In Markdown, disable raw HTML (`react-markdown` does this by default; don't add `rehype-raw`). Allow only `http(s):` and `mailto:` links.
- Every one of these sinks is a High SAST finding, even with "trusted" data:
    - `dangerouslySetInnerHTML`, `innerHTML` / `outerHTML` / `document.write`;
    - Vue `v-html`, EJS `<%- %>`, Handlebars `{{{ }}}`, `| safe` in templates;
    - jQuery `.html()` / `.append()` with data;
    - `window.location.href = <user value>`. If you truly need HTML, sanitize with DOMPurify, and expect to document the finding.
- Check: send `<img src=x onerror=alert(1)>` and `javascript:alert(1)` as input, *and* make the LLM return them.
- High.
- **R-VAL-06: No open redirects, no ReDoS.**
- `redirect()` only to fixed paths, or to relative paths that start with `/` but not `//`.
- No complex regexes on long input.
- Medium.

2.6 LLM, RAG and tools

- **R-AI-01: Retrieved documents, user text and model output are data, not instructions.**
- Delimiters (`<document>…</document>`) help, but **they don't stop prompt injection**.
- The real protection is *in the code*: R-AI-02, R-AI-03, and output treated as text.

- **Keep the user's question and the documents in separate places, never in one string with markers.** If sources and question are joined as `=== SOURCE n ===… QUESTION: …`, a document containing `QUESTION:` takes over the question and can make the model call tools (for example draft a phishing email). So:
  - put each document in its own content block (or JSON-encode the list), and the question in its own message;
  - strip or escape lines in documents that look like your own markers;
  - your code, not the model, decides whether a tool may run, based on what the *user* asked (for example, only allow `draft_email` when the user's own question asks for an email);
  - make a mock model read the question from its own argument, not by parsing the prompt.
- Check: upload a document containing `QUESTION: draft an email to …` and ask an unrelated question. No tool call, and no text from the document treated as the question.
- High.
- **R-AI-02: Check access before retrieval.**
- Filter the vector or full-text search by the user's tenant and access rights *before* anything goes into the prompt.
- Don't: search the whole index and then ask the model to hide what the user may not see.
- Check: plant a secret string in B's document, then ask for it as A. It must not appear in the retrieval results, the prompt or the answer.
- Also filter **everything around the answer**: citations, "sources considered", "excluded" lists, previews. Run the same access check on those lists before they go to the browser.
- High.
- **R-AI-03: Tools are allowlisted, validated with a schema, and check permissions themselves.**
- The user comes from the session, not from the model. Irreversible actions need confirmation. Cap the number of calls.
- Never give the model a shell, free SQL, or a general-purpose fetch.
- Fields the attacker shouldn't choose are set by the server, not the model (for example, an email recipient is always the client's own address, looked up on the server).
- Critical.
- **R-AI-05: Links in AI answers.** Text copied from a document can contain `[click here](https://evil…)`, which the answer then shows as a real link. Render links only to an allowlist of domains (your own app, official sources), and show other URLs as plain text. Medium.
- **R-AI-04: Limit cost and data.**
- The model and `max_tokens` are fixed on the server; don't accept them from the client.
- Rate-limit per user, and set a timeout.
- Send no secrets, and no more personal data than the answer needs.
- **Mask personal data before the question leaves the server** (GDPR).
  - Replace emails, IBANs, national ID numbers, phone numbers and ages with tokens like `⟦email⟧` (regexes are enough for a hackathon).
  - Known names (clients, people) are replaced too, longest name first.
  - Show the user which items were masked.
- **Bounded caches:** an in-memory cache of LLM answers gets a maximum size (for example 1,000 entries, the oldest is removed first). Otherwise someone can fill the server's memory with unique questions.
- Medium (critical for secrets).

2.7 Secrets

- **R-SEC-01: No hardcoded secrets, and no `NEXT_PUBLIC_` on a secret.** (secrets scanner; Next.js: only `NEXT_PUBLIC_` goes to the client)

- Do: real keys in Vercel env vars; `.env.local` in `.gitignore`; `.env.example` with placeholders; `import "server-only"` in every file that reads `process.env` secrets.
- Check:
  - `git ls-files | grep -E '(^|/)\.env|\.pem$|\.key$'` shows only `.env.example`;
  - `grep -rn "NEXT_PUBLIC_" src`;
  - optionally `gitleaks git .` (also scans git history).
- If the case uses **Vite** (not Next.js): never `loadEnv(mode, cwd, "")` with an empty prefix, and never `define: { "process.env": process.env }`. Both put every env var in the browser bundle (critical).
- Critical.
- **R-SEC-02: A key that was ever committed must be rotated.** Deleting it doesn't help, because it stays in the history. Critical.

2.8 Dependencies and licenses

- **R-DEP-01: One committed lockfile, few well-known packages, `npm ci`.** (Aikido scans lockfiles) Aikido only scans dependencies through a lockfile it recognizes: `package-lock.json`, `pnpm-lock.yaml`, `yarn.lock` or `bun.lock` for JS; `requirements.txt` (pinned with `==`), `poetry.lock`, `uv.lock` or `Pipfile.lock` for Python. Without one, it can't check your dependencies. Commit exactly one per project.
- Check: `npm audit --audit-level=high`.
- Watch out for typosquats: package names that look almost like popular ones.
- **A high advisory with no fix, only in dev tooling** (for example `eslint-config-next` → `braces`): `npm audit --omit=dev` must be clean. Document the dev-only one in the README, and ignore it per finding in Aikido with the reason "dev-only lint tooling, not in the runtime bundle". Don't remove ESLint for it.
- High.
- **R-DEP-02: Check licenses** (GPL/AGPL) in Aikido's license report, and remove unused packages. (Aikido does license scanning) Medium.

2.9 HTTP, CSRF, errors

- **R-HTTP-01: Set security headers in `next.config.ts`** (section 5).
- **Merge** them into the `next.config.ts` that `create-next-app` made. Don't replace the file, or you lose its existing settings (such as the Tailwind turbopack rule).
- **The `/:path*` entry overwrites stricter headers set in route handlers.** (A route that sets `default-src 'none'; sandbox` on downloads still gets the global CSP.) Put the strict CSP for downloads and share links in a *second* entry **later** in the `headers()` list; per the Next.js docs, the last matching entry wins.
- The page CSP has no `script-src` (that would break Next.js without nonces), so it doesn't stop XSS. List it as a trade-off; a nonce CSP is the upgrade.
- Also set `poweredByHeader: false`.
- Check: `curl -sI https://<app>`.
- Medium.
- **R-HTTP-02: CSRF and CORS.** (Next.js)
- **Server actions** already compare `Origin` with `Host` themselves.
- **Route handlers** don't. Every POST, PUT, PATCH or DELETE route handler with cookie auth checks that `Origin` matches the `Host` header, just as Next.js does (section 5). Compare against the `Host` header, not `request.url`: in `next start`, `request.url` says `localhost` even when the browser opened `127.0.0.1`, so the app's own buttons got 403.
- No GET requests that change data.
- Never `Access-Control-Allow-Origin: *` together with cookies.
- High.

- **R-HTTP-03: Generic errors with the correct status code.**
- 400 invalid input, 401 not logged in, 403/404 not allowed, 413 too large, 429 too many requests, 503 a store is down, 500 anything else.
- Never send `err.message`, a stack trace or SQL text to the client.
- Medium.

2.10 Uploads and downloads

- **R-FILE-01: Validate uploads.**
- Size limit *before* buffering the whole file.
- Allowlist of types, checked on the file's actual content (magic bytes).
- File name generated on the server.
- Private storage (Vercel Blob or S3), not `public/`.
- Don't trust `file.type` or the extension.
- **The stored name follows the detected type.** Checking only the content and keeping the user's name lets through names like `Invoice Q2.html` (a PDF/HTML mix) and .(retcarahc tfel-ot-thgir neddih a) `exe.txt` sretcarahc idib dna lortnoc pirts dna ,(`txt.` , `fdp.`) epyt detceted eht hctam ot noisnetxe eht ecroF .seman morf (9602+U–6602+U ,E202+U–A202+U ,F/E002+U ,F900+U–F700+U ,F100+U–0000+U)
- High.
- **R-FILE-03: Public share links.**
- The token is 32 random bytes (`randomBytes(32).toString("base64url")`). Store only its **SHA-256 hash**. On open, compare hashes.
- The link has an expiry (7 days), is checked on *every* open, and can be revoked. Only the creator (or an admin of the same tenant) can revoke, and a revoke is atomic.
- On every open, re-check that the creator still has access to the document.
- Rate-limit opens per IP. Send `Referrer-Policy: no-referrer` and `X-Robots-Tag: noindex`. Serve the file as an attachment. An unknown, revoked or expired token gives the same 404.
- **R-FILE-02: Check permission on every download.**
- Do: check owner or tenant, then use a signed URL or `Content-Disposition: attachment` together with `nosniff`.
- Never serve uploaded HTML or SVG from the app's own domain.
- High.

2.11 Infra, CI and repository settings

- **R-IAC-00: Don't add infra files you don't need.** Aikido has hundreds of IaC rules (AWS, GCP, Azure, Docker, Kubernetes, Terraform, Pulumi, Ansible). Every Dockerfile, `k8s/*.yaml`, Terraform or Pulumi file in the repo gets scanned. On Vercel you need none of them. Don't let an agent generate a "production-ready" Dockerfile or Kubernetes manifest "just in case": each one brings its own findings. If the case does need one, R-IAC-01 applies.
- **R-IAC-01: Docker**
- Pin the base image (`node:22.11-alpine@sha256:…`), never `latest` (High).
- Run as a non-root `USER` (Medium).
- Use a multi-stage build and a `.dockerignore` that excludes `.env`. No secrets in image layers.
- No `apk add --allow-untrusted`. No `curl … | sh` or downloads without a checksum check.
- Kubernetes, if any: `runAsNonRoot: true`, `readOnlyRootFilesystem: true`, `allowPrivilegeEscalation: false`, `privileged: false`, drop `NET_RAW`.
- Medium–High.
- **R-IAC-02: GitHub Actions**

- Set `permissions: contents: read` at the top of the workflow; widen it per job only when needed.
- Pin third-party actions by full commit SHA (High).
- No `pull_request_target` or `workflow_run` triggers (High).
- Never put `${{ github.event.* }}` (titles, branch names, comments) directly in a `run:` script. Pass it through `env:` instead (template injection, High).
- `actions/checkout` with `persist-credentials: false` unless the job pushes.
- No downloaded binaries without a checksum.
- High.
- **R-IAC-04: GitHub repository settings, which Aikido also scores.** (rules `github_001`–`github_009`)
- **Plan limit (checked 18 October 2026):** on a free GitHub account, branch protection and rulesets only work on **public** repos. On a private repo, GitHub answers "Upgrade to GitHub Pro or make this repository public". So either make the hackathon repo public before the final scan (most hackathons require that anyway, but first double-check that no secrets are in the history, R-SEC-01/02), or list the missing protection as a trade-off.
- **Before the final scan**, turn on branch protection (a ruleset) for `main`:
  - require a pull request before merging (`github_006`, High);
  - require at least 1 approving review (`github_002`, Medium);
  - require a passing status check, such as lint/test/build in GitHub Actions (`github_004`, Medium);
  - dismiss stale approvals when new commits are pushed (`github_007`, Low);
  - require linear history (`github_005`, Low).
- If the repo is in a GitHub **organization**: enforce 2FA for members (`github_001`, Critical), set the default member permission to "No permission" (`github_003`, High), and optionally an IP allow list (`github_008`, Medium, often not possible on a free plan; list it as a trade-off).
- Hackathon tip: work on `main` during the build, and switch these rules on in the last 20 minutes. From then on, every fix goes through a pull request with one review from a teammate. Add a tiny CI workflow (R-IAC-02 rules) that runs `npm ci && npm run lint && npm test && npm run build`, so the status check exists.
- **R-IAC-03: Vercel**
- Use separate dev and prod secrets, turn on preview protection if previews show sensitive data, and turn off debug in production.
- Zen (Aikido's runtime firewall) is extra protection only, and replaces no checks in the code.
- **R-IAC-05: Mobile apps (only if the case is a mobile app).**
- Android: `android:debuggable="false"`, `android:allowBackup="false"`, an explicit `android:exported` on every component (`false` unless it must be launched by other apps), a `networkSecurityConfig` that forbids cleartext, permissions on content providers, and no autocomplete on password fields.
- iOS (`Info.plist`): no `NSAllowsArbitraryLoads` or other HTTP exceptions, minimum TLS 1.2+, keep forward secrecy, no local networking in release builds.
- Expo/React Native: check the generated native config, not only `app.json`.

2.12 Logging, privacy, repo

- **R-LOG-01: Never log passwords, tokens, cookies, headers, full bodies or full prompts.** Log only the event, a pseudonymous user ID and the outcome. Use synthetic demo data, not real personal data. Medium.
- **R-HYG-01: Keep the repo clean.** (the secrets scanner flags strings that look random)
- Keep `.next/`, `node_modules/`, coverage and `.env*` out of git.
- Remove `/api/debug`, `TODO bypass auth` and dead code.
- Use obviously fake test values (`test-key-not-real`), never realistic-looking random tokens.
- Medium.

- **R-TEST-01: Automate the negative tests** in section 4: no cookie, user A requests B's data, a forged role, mass assignment, CSRF, a replayed request, concurrent requests, XSS, an oversized upload, prompt injection. A clean build plus a demo that works is not proof that the app is secure. High.
- Write these unit tests in `src/lib/security.test.ts` (`node --test`). Aim for 10–15:
  - a signed session works and expires;
  - a tampered or forged session is refused;
  - logout also stops a copied cookie;
  - changing a password ends sessions;
  - a correct password works, and a wrong one or an unknown user fails in the same time;
  - 20 parallel login attempts: at most 5 are evaluated per account+IP;
  - the correct password still works after 10 wrong guesses (refund);
  - `x-forwarded-for` is ignored unless `VERCEL` is set;
  - cross-site requests are refused;
  - users reach only their own tenant's objects;
  - 20 parallel transfers never overspend;
  - `setNx` only writes the first value;
  - production without a store gives 503.
- Files with `import "server-only"` can't be imported by a plain Node test. Run tests with `node --conditions=react-server --import tsx --test` (install `tsx` as a dev dependency).
- In production mode the cookie is `__Host-session` with `Secure`, which curl doesn't send back over plain `http://localhost`. In test scripts, read the `Set-Cookie` header and send it yourself as a `cookie:` header.
- **Keep an attack script in the repo** (`scripts/attack-tests.mjs`) that runs the section 4 checks against `next start`. Aim for 60+ checks; they catch regressions.
- **R-DOC-01: README `## Security` section.** It maps each control to its file and test, and lists accepted trade-offs (section 6). Only describe what's actually implemented. The README helps the reader; it doesn't fix anything and it doesn't silence a finding.

Section 3: Agent contract (paste into CLAUDE.md)

```
# Security rules (Aikido-proof): follow these while writing code, not afterwards
Treat every request as an attacker. Never hide, suppress or relabel vulnerabilities to improve a score: fix them.

1. Before coding: decide whether the app needs a login (no login only if everything is public, the same for everyone, and nobody can change what others see; otherwise login plus a read-only guest mode). Then list the roles, resources, owner/tenant boundaries, public endpoints, mutations, LLM tools, uploads and secrets. Decide who may do what. Without a login, every route must be safe for anyone: rate limits per IP plus a global cap, validation, no personal data.
2. Every route handler, server action, data-loading page and data-access function calls requireUser() itself and checks permission. proxy.ts and hidden UI are NOT authorization.
3. The user's identity comes ONLY from the session. NEVER read userId, role, tenantId or isAdmin from the body, the query string, route params, headers or LLM output.
4. Every lookup, update, delete, download, export and search filters on the owner/tenant from the session. Deny by default. On a mismatch return 404. For writes, repeat the condition inside the atomic write.
5. Votes, credits and one-time actions are atomic (Redis SET NX, transaction, unique constraint) and idempotent. No check-then-write. A change to several records (buy, transfer, approve + points) is ONE atomic step: a Lua script, compare-and-set on one tenant document, or a DB transaction. Never a lock with a TTL around several calls.
5b. Re-check every business rule at EVERY step that commits it, especially the final approval: the rule itself, date/time still valid, both parties still active members of the tenant.
5c. Anything a new account gets for free (credit, votes, rankings) is multiplied by fake sign-ups: free credit can only be spent, not moved; votes and rankings only count for seeded or verified accounts, or are listed as a trade-off. Never derive a public name from an email.
6. Every input (body, params, searchParams, headers, webhooks, tool arguments) goes through a strict zod schema with .strict(), min/max and a body size limit enforced while reading. Copy only allowed fields. Prices, roles, flags and workflow state are decided on the server.
7. No SQL/NoSQL string concatenation, no eval/new Function/exec/shell:true, no file paths from user input, no merging untrusted objects.
8. No server-side fetch of a URL from a user, a document or the LLM. Use fixed endpoints only.
9. Render user and LLM content as text. No dangerouslySetInnerHTML, no raw HTML in Markdown, only http(s)/mailto links.
10. Sessions: signed or opaque cookie with HttpOnly, Secure, SameSite and an expiry, revoked on the server at logout. Passwords hashed with scrypt/Argon2id. If the session store or limiter fails: 503, never guest or admin access. In production without the shared store: 503 unless ALLOW_MEMORY_STORE=true. Check config on use, never at import (the build must not crash).
11. Route handlers that change data check that Origin matches the Host header (not request.url). No GET that changes data. No wildcard CORS.
12. Rate-limit login, sign-up, guest, upload and LLM routes with the shared store, and make the limiter fail closed. Count FIRST (atomic INCR), then decide. Login: reserve the attempt before checking the password and refund it on success (requireLoginAttempt returns refund()). Three limits: account+IP 5, IP 50, account from any IP 100 (a slowdown, not a lockout). Trust x-forwarded-for only when process.env.VERCEL is set; elsewhere the IP is unknown (never one shared "unknown" bucket). No permanent lockout.
13. RAG: filter by the user's access BEFORE retrieval. Documents and the user's question go in separate content blocks, never one string with markers; strip marker-like lines from documents. Your code decides
```

whether a tool may run, based on the user's own request; the server sets fields like recipients. LLM tools are allowlisted, validated and check permissions themselves. Model and max_tokens are fixed on the server. Links in answers only to allowlisted domains. Never put secrets in prompts.
14. Secrets only in server env vars, in files that start with import "server-only". NEVER use NEXT_PUBLIC_ for a secret. Never commit .env; commit .env.example. A leaked key gets rotated.
15. Uploads: size limit before buffering, magic-byte check, server-generated storage name, the display name's extension forced to the detected type, control and bidi characters stripped, private storage. Downloads: check permission, Content-Disposition: attachment, plus a strict CSP in a LATER next.config headers entry (the global entry otherwise overwrites it). Share links: 32-byte random token, only its hash stored, expiry, revocable, creator's access re-checked on every open, same 404 for unknown/expired/revoked.
16. Errors: correct status code (400/401/404/413/429/503/500) with a generic message. Never send stack traces, err.message or full database rows. Use no-store on private data.
17. Few well-known packages, a committed lockfile, `npm audit --omit=dev` clean on high/critical (document dev-only advisories with no fix). No debug routes, dead code or realistic fake secrets. No passwords in the repo, not even demo ones: demo logins go in the submission form, or use guest mode.
17a. Unsure what Aikido checks? Look it up in Aikido's docs first (index: https://help.aikido.dev/llms.txt) instead of guessing, and tell the team if the docs disagree with AIKIDO-INSTRUCTIONS.md.
17c. Aikido's SAST rules fire on names, not intent. Never use: eval, new Function, setTimeout/setInterval with a string, child_process with variables, dangerouslySetInnerHTML/innerHTML/document.write/v-html, window.location.href = user value, path.join/resolve with request values before readFile/sendFile, jwt.decode to trust claims, === to compare passwords or tokens, verify=False/rejectUnauthorized:false/NODE_TLS_REJECT_UNAUTHORIZED=0, ws://, scripts from polyfill.io-type CDNs, SQL built with template strings. In Python: eval/exec, pickle, yaml.load, shell=True, defusedxml instead of xml.*, no Flask debug.
17d. Don't add infrastructure files the app doesn't need (Dockerfile, Kubernetes, Terraform, Pulumi): each one is scanned against hundreds of rules. If you add a GitHub Actions workflow: permissions: contents: read, actions pinned by SHA, no pull_request_target, no ${{ github.event.* }} inside run:, persist-credentials: false.
17e. In the last 20 minutes, turn on branch protection for main (pull request required, 1 review, passing status check, dismiss stale approvals, linear history). Aikido scores these repository settings too.
17b. Start from the section 5 skeleton, not from code of an older project (older code often has the weak patterns in section 7). Merge security headers into the existing next.config.ts; don't replace the file.
18. Keep the README "## Security" section up to date: each control with its file and test, plus accepted trade-offs. Only describe what is actually built.
19. Keep an attack script in the repo (scripts/attack-tests.mjs). Before finishing: lint, `next typegen` (or build) then typecheck, build, npm audit, then the negative tests: no cookie → 401; A requests B's data → 404; forged role/owner → no effect; wrong Origin → 403; 20 parallel identical requests → 1 change; XSS payload → shown as text; prompt injection (including a document containing your own prompt markers) → no extra data or tool calls; 25 wrong logins with rotating x-forwarded-for → 429; two requests that are fine alone but break a rule together → second refused; 20 parallel buys/transfers → totals still add up.

Section 4: Final checklist (step 5, last 30 minutes)

Minutes 0–5: repo and dependencies

- [ ] `git status --short && git rev-parse HEAD`: is this the commit Aikido scans?
- [ ] `git ls-files | grep -E '(^|/)(\.env(\.|$)|node_modules/|\.next/|coverage/)|\.(pem|key)$'` shows only `.env.example`.
- [ ] `git ls-files | grep -iE 'dockerfile|\.tf$|k8s|helm|pulumi|\.github/workflows'`: every infra file is needed, and follows R-IAC-01/02.
- [ ] `npm audit --omit=dev --audit-level=high` is clean (dev-only advisories documented). Then run `npm run lint`, `npx next typegen && npx tsc --noEmit` and `npm run build`.
- [ ] Optional: `gitleaks git .`

Minutes 5–10: grep for dangerous patterns (not every hit is a bug, but look at each one)

```
grep -rnE 'NEXT_PUBLIC_.*(KEY|SECRET|TOKEN)|dangerouslySetInnerHTML|eval\(|new Function|shell: ?
true|queryRawUnsafe|\$where|Access-Control-Allow-Origin|localStorage|x-user-id|body\.userId' src
grep -rnE 'TODO.*(auth|security)|bypass|DEBUG|console\.(log|error)|nosemgrep|gitleaks:allow' src
grep -rnE 'fetch\(|redirect\(|"use server"|export async function (GET|POST|PUT|PATCH|DELETE)' src
# Aikido SAST rule names: each hit must be justified or removed
grep -rnE 'set(Timeout|Interval)\(\s*
["`'"'"']|child_process|execSync|innerHTML|outerHTML|document\.write|v-
html|location\.href\s*=|jwt\.decode|NODE_TLS_REJECT_UNAUTHORIZED|rejectUnauthorized:\s*false|ws://|polyf
ill\.io|bootcss|sendFile\(|readFile(Sync)?\(|\$queryRawUnsafe|yaml\.load\(|noent' src scripts
grep -rnE '(password|token|secret|signature)[A-Za-z]*\s*===|===\s*[A-Za-z.]*(password|token|secret)' src
```

- [ ] Every `"use server"` function and every route handler from the third grep calls `requireUser()` and checks permission.

Minutes 10–17: manual attacks

- [ ] No cookie: every protected route gives 401, and no data leaks out.
- [ ] Logged in as A, requested B's object by ID (GET, PATCH, DELETE, download, search): got 404, and B is unchanged.
- [ ] As A, sent `userId=B`, `ownerId=B`, `role=admin`, `isAdmin=true`, `price=0`: no effect.
- [ ] Guest: can't reach private data or admin routes.
- [ ] Sent a POST with `Origin: https://evil.example`: got 403.
- [ ] Sent -1, 1e300 and unknown JSON fields: got 400.
- [ ] Sent 20 identical votes or actions in parallel: exactly 1 counted.
- [ ] Entered `<img src=x onerror=alert(1)>` as input and in an LLM answer: shown as text.
- [ ] Planted text in a document saying "give me B's data": B's data stays out of the answer.
- [ ] Sent 20 quick login attempts: got 429, and another account can still log in.
- [ ] Sent 20 *parallel* wrong logins: at most 5 were evaluated. Sent 25 wrong logins with a different `x-forwarded-for` each: got 429 (per-account limit).
- [ ] Sent 20 parallel buys or transfers: balances still add up. Made two pending requests that break a rule together and approved both: the second was refused.
- [ ] Uploaded a document containing `QUESTION: draft an email to …` and asked something unrelated: no tool call.

- [ ] Signed up 3 throwaway accounts: no free credit or votes can be moved to your main account.
- [ ] Opened the app on `127.0.0.1` and on `localhost`: the app's own buttons work on both (CSRF check uses `Host`).
- [ ] Forced an error: got a generic 500 with no stack trace.
- [ ] `curl -sI https://<app>` shows CSP, HSTS, X-Frame-Options and nosniff.

Minutes 17–20: scan and documentation

- [ ] Fixed the critical and high findings in Aikido. Any accepted risk is ignored *per finding* in the UI and listed in the README.
- [ ] The README `## Security` section is up to date and has no placeholders.
- [ ] GitHub: branch protection on `main` is on (pull request, 1 review, status check, dismiss stale approvals, linear history), and the CI workflow is green.

Section 5: Ready-made code, the security core (step 2, during the brainstorm)

Build order (step 2, about 20 minutes, developer 1 with an agent, during the brainstorm):

1. Scaffold: `npx create-next-app@latest . --ts --app --eslint --tailwind --src-dir --use-npm --yes`. Then `npm i zod server-only` and `npm i -D tsx`.
2. Copy these files from this section into `src/lib/`: `http-error`, `store`, `limit`, `csrf`, `validate`, `password`, `users`, `auth`, `access`. Then copy `src/proxy.ts` and the login and logout routes.
3. Merge the headers into the scaffold's `next.config.ts`, and extend `.gitignore` and `.env.example`.
4. Run `node scripts/create-users.mjs …` and put the output in `.env.local` (plus `SESSION_SECRET` from `openssl rand -base64 48`).
5. Before writing features: list roles, tenants, objects and the money/vote flows (section 3, rule 1). Every new route follows the example route: Origin check → `requireUser()` → rate limit → schema → access check → one atomic write → minimal response.
6. Write `src/lib/security.test.ts` and `scripts/attack-tests.mjs` *while* building, not at the end.

Requires `npm install zod server-only` (and `npm install -D tsx` for tests), plus the env vars `SESSION_SECRET` (at least 32 random bytes) and, in production, `UPSTASH_REDIS_REST_URL` and `UPSTASH_REDIS_REST_TOKEN` (no Upstash package needed: the store talks REST). All snippets below are typechecked and covered by behavior tests.

`src/lib/http-error.ts`: safe errors

```
import { ZodError } from "zod";

export class HttpError extends Error {
  constructor(public readonly status: 400 | 401 | 403 | 404 | 409 | 413 | 429 | 503, public readonly
publicMessage: string) {
    super(publicMessage);
  }
}

export function safeError(error: unknown): Response {
  if (error instanceof HttpError) {
    return Response.json({ error: error.publicMessage }, { status: error.status, headers: { "Cache-
Control": "no-store" } });
  }
  if (error instanceof ZodError || error instanceof SyntaxError) {
    return Response.json({ error: "Invalid request" }, { status: 400 });
  }
  console.error("Unhandled application error"); // no err.message, stack, SQL or provider response
  return Response.json({ error: "Internal server error" }, { status: 500 });
}
```

`src/lib/password.ts`: scrypt, constant time, no user enumeration

```
import "server-only";
import { randomBytes, scrypt as scryptCb, timingSafeEqual } from "node:crypto";
import { promisify } from "node:util";

const scrypt = promisify(scryptCb) as (password: string, salt: string, keylen: number) =>
Promise<Buffer>;
export type PasswordRecord = { salt: string; hash: string };

export async function hashPassword(password: string): Promise<PasswordRecord> {
  const salt = randomBytes(16).toString("hex");
  return { salt, hash: (await scrypt(password, salt, 32)).toString("hex") };
}

// Unknown users are hashed against a dummy, so the response time doesn't reveal who exists.
const DUMMY: PasswordRecord = { salt: "0".repeat(32), hash: "0".repeat(64) };

export async function verifyPassword(password: string, stored: PasswordRecord | null): Promise<boolean>
{
  const { salt, hash } = stored ?? DUMMY;
  const actual = await scrypt(password.slice(0, 200), salt, 32);
  const expected = Buffer.from(hash, "hex");
  return actual.length === expected.length && timingSafeEqual(actual, expected) && stored !== null;
}

// Stored in the session: a new password (new salt) makes every older session invalid.
export const passwordVersion = (record: PasswordRecord) => record.salt.slice(0, 8);
```

`src/lib/users.ts`: where users come from (replace with your data)

```
import "server-only";
import { passwordVersion, type PasswordRecord } from "./password";

export type AppUser = { id: string; tenantId: string; role: "guest" | "user" | "admin"; passwordVersion:
string };
type Row = { user: AppUser; password: PasswordRecord; active: boolean };

// Demo users: AUTH_USERS="id:tenantId:role:salt:hash,..." from scripts/create-users.mjs. Never plain
passwords.
function rows(): Map<string, Row> {
  const map = new Map<string, Row>();
  for (const entry of (process.env.AUTH_USERS ?? "").split(",")) {
    const [id, tenantId, role, salt, hash] = entry.trim().split(":");
    if (!id || !tenantId || !salt || !hash || !["user", "admin"].includes(role)) continue;
    const password = { salt, hash };
    map.set(id, { user: { id, tenantId, role: role as AppUser["role"], passwordVersion:
passwordVersion(password) }, password, active: true });
  }
  return map;
```

```
}

export async function findLoginRecord(username: string): Promise<{ user: AppUser; password:
PasswordRecord } | null> {
  const row = rows().get(username);
  return row && row.active ? { user: row.user, password: row.password } : null;
}

// Called on EVERY request: users who left, were deleted or changed password are refused (R-AUTH-06).
export async function findActiveUser(id: string): Promise<AppUser | null> {
  const row = rows().get(id);
  return row && row.active ? row.user : null;
}
```

`src/lib/auth.ts`: signed session with logout revocation

```
import "server-only";
import { createHmac, randomBytes, timingSafeEqual } from "node:crypto";
import { cookies } from "next/headers";
import { HttpError } from "./http-error";
import { store } from "./store";
import { findActiveUser, type AppUser } from "./users";

export type SessionUser = AppUser;
const PROD = process.env.NODE_ENV === "production";
export const SESSION_COOKIE = PROD ? "__Host-session" : "session";
const MAX_AGE_S = 8 * 60 * 60;
type Session = { sub: string; sid: string; pv: string; exp: number };

// Checked on use, not at import: a missing secret gives 503 instead of crashing `next build`.
function secret(): string {
  const s = process.env.SESSION_SECRET ?? "";
  if (s.length < 32) throw new HttpError(503, "Login is not configured");
  return s;
}
const sign = (data: string) => createHmac("sha256", secret()).update(data).digest("base64url");
// Constant-time compare for anything secret-like (scanners flag `===` on password/token values).
const safeEqual = (a: string, b: string) => {
  const x = Buffer.from(a), y = Buffer.from(b);
  return x.length === y.length && timingSafeEqual(x, y);
};

// Call only AFTER the password was verified on the server.
export async function startSession(user: AppUser) {
  const body: Session = { sub: user.id, sid: randomBytes(16).toString("base64url"), pv:
user.passwordVersion, exp: Math.floor(Date.now() / 1000) + MAX_AGE_S };
  const payload = Buffer.from(JSON.stringify(body)).toString("base64url");
  (await cookies()).set(SESSION_COOKIE, `${payload}.${sign(payload)}`, {
    httpOnly: true, secure: PROD, sameSite: "strict", path: "/", maxAge: MAX_AGE_S,
  });
```

```
}

function readToken(token: string | undefined): Session | null {
  if (!token || token.length > 2048) return null;
  const [payload, sig, extra] = token.split(".");
  if (!payload || !sig || extra !== undefined) return null;
  if (!safeEqual(sig, sign(payload))) return null;
  try {
    const s = JSON.parse(Buffer.from(payload, "base64url").toString("utf8"));
    if (typeof s.sub !== "string" || typeof s.sid !== "string" || typeof s.pv !== "string" || typeof
s.exp !== "number") return null;
    return s.exp * 1000 > Date.now() ? (s as Session) : null;
  } catch {
    return null;
  }
}

export async function currentUser(): Promise<AppUser | null> {
  const session = readToken((await cookies()).get(SESSION_COOKIE)?.value);
  if (!session) return null;
  // Store errors are NOT caught: they become a 503, never "logged in" (fail closed).
  if (await store().get(`revoked:${session.sid}`)) return null;
  const user = await findActiveUser(session.sub);
  return user && safeEqual(user.passwordVersion, session.pv) ? user : null;
}

export async function requireUser(opts: { allowGuest?: boolean } = {}): Promise<AppUser> {
  const user = await currentUser();
  if (!user || (user.role === "guest" && !opts.allowGuest)) throw new HttpError(401, "Authentication
required");
  return user;
}

// Logout: the session id stays revoked until the cookie would have expired anyway, so a copied cookie
stops working.
export async function endSession() {
  const jar = await cookies();
  const session = readToken(jar.get(SESSION_COOKIE)?.value);
  if (session) await store().setNx(`revoked:${session.sid}`, "1", session.exp * 1000 - Date.now());
  jar.delete(SESSION_COOKIE);
}
```

This is the complete pattern; use it as is. Alternative: an opaque random token whose hash is stored in Redis. This makes role changes immediate, but needs Redis on every request. Both pass, as long as they're signed or random, expire, and are revoked at logout.

`scripts/create-users.mjs`: demo users without passwords in the repo

```
// Usage: node scripts/create-users.mjs anna:hospital-1:user marc:hospital-1:admin
// Prints the AUTH_USERS line for .env.local / Vercel (stdout) and the generated passwords (stderr).
// Keep the passwords in TEST-ACCOUNTS.local.md (gitignored) or your password manager, never in the
```

```
repo.
import { randomBytes, scryptSync } from "node:crypto";

const entries = [];
for (const spec of process.argv.slice(2)) {
  const [id, tenantId, role] = spec.split(":");
  const password = randomBytes(9).toString("base64url");
  const salt = randomBytes(16).toString("hex");
  entries.push([id, tenantId, role, salt, scryptSync(password, salt, 32).toString("hex")].join(":"));
  console.error(`${id}\t${password}`);
}
console.log(`AUTH_USERS=${entries.join(",")}`);
```

`src/app/api/auth/login/route.ts` and `logout/route.ts`

```
import { z } from "zod";
import { startSession } from "@/lib/auth";
import { requireSameOrigin } from "@/lib/csrf";
import { requireLoginAttempt } from "@/lib/limit";
import { verifyPassword } from "@/lib/password";
import { findLoginRecord } from "@/lib/users";
import { validatedJson } from "@/lib/validate";
import { HttpError, safeError } from "@/lib/http-error";

const schema = z.object({ username: z.string().trim().toLowerCase().min(1).max(254), password:
z.string().min(1).max(200) }).strict();

export async function POST(request: Request) {
  try {
    requireSameOrigin(request);
    const { username, password } = await validatedJson(request, schema);
    const refund = await requireLoginAttempt(username, request); // reserved BEFORE the password check
    const record = await findLoginRecord(username);
    if (!record || !(await verifyPassword(password, record.password))) {
      throw new HttpError(401, "Wrong username or password"); // same message for unknown users
    }
    await refund(); // a correct password doesn't count against the limits
    await startSession(record.user);
    return Response.json({ ok: true }, { headers: { "Cache-Control": "no-store" } });
  } catch (e) { return safeError(e); }
}
```

```
// src/app/api/auth/logout/route.ts
import { endSession } from "@/lib/auth";
import { requireSameOrigin } from "@/lib/csrf";
import { safeError } from "@/lib/http-error";

export async function POST(request: Request) {
  try {
    requireSameOrigin(request);
```

```
    await endSession();
    return Response.json({ ok: true });
  } catch (e) { return safeError(e); }
}
```

Note: `verifyPassword` also runs for unknown users, with a dummy hash, so timing doesn't reveal which usernames exist.

`src/proxy.ts`: first gate, deny by default

```
import { NextResponse, type NextRequest } from "next/server";

// Only these paths work without a session. Everything else needs one, and is checked AGAIN in the
route.
const PUBLIC = [/^\/login$/, /^\/api\/auth\/(login|signup|guest)$/, /^\/s\/[\w-]+$/];

export function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl;
  if (PUBLIC.some((re) => re.test(pathname))) return NextResponse.next();
  if (request.cookies.has("__Host-session") || request.cookies.has("session")) return
NextResponse.next();
  if (pathname.startsWith("/api/")) return NextResponse.json({ error: "Authentication required" }, {
status: 401 });
  return NextResponse.redirect(new URL("/login", request.url));
}

export const config = { matcher: ["/((?!_next/static|_next/image|favicon.ico).*)"] };
```

The proxy only checks that a cookie *exists*. Signature, revocation, user and permission checks happen in every route via `requireUser()` (R-AUTHZ-01).

`src/lib/access.ts`: ownership check

```
import "server-only";
import { HttpError } from "./http-error";
import type { SessionUser } from "./auth";

type Owned = { tenantId: string; ownerId: string };

// For reads. For writes: repeat this condition INSIDE the atomic write.
export function requireAccess<T extends Owned>(user: SessionUser, obj: T | null): T {
  if (!obj || obj.tenantId !== user.tenantId || (obj.ownerId !== user.id && user.role !== "admin")) {
    throw new HttpError(404, "Not found"); // 404 so nobody can find out the object exists
  }
  return obj;
}
```

`src/lib/validate.ts`: JSON size-limited while reading, then zod

```
import { z } from "zod";
import { HttpError } from "./http-error";

export async function validatedJson<T extends z.ZodType>(request: Request, schema: T, maxBytes =
16_384): Promise<z.infer<T>> {
  if (request.headers.get("content-type")?.split(";")[0] !== "application/json") throw new
HttpError(400, "Expected JSON");
  const reader = request.body?.getReader();
  if (!reader) throw new HttpError(400, "Missing body");
  const chunks: Uint8Array[] = [];
  let size = 0;
  for (;;) {
    const { done, value } = await reader.read();
    if (done) break;
    size += value.byteLength;
    if (size > maxBytes) { await reader.cancel(); throw new HttpError(413, "Request too large"); }
    chunks.push(value);
  }
  const bytes = new Uint8Array(size);
  let offset = 0;
  for (const c of chunks) { bytes.set(c, offset); offset += c.byteLength; }
  return schema.parse(JSON.parse(new TextDecoder("utf-8", { fatal: true }).decode(bytes)));
}
```

`src/lib/store.ts`: Upstash over REST, memory locally, 503 in production without it

```
import "server-only";
import { HttpError } from "./http-error";

export interface Store {
  /** INCR + expiry on first hit; returns the new value. Atomic. */
  count(key: string, windowMs: number): Promise<number>;
  /** DECR: hands back one counted attempt (e.g. after a correct password). */
  uncount(key: string): Promise<void>;
  /** SET key value NX (optionally expiring); true if this call created it. Atomic. */
  setNx(key: string, value: string, ttlMs?: number): Promise<boolean>;
  get(key: string): Promise<string | null>;
  /** Moves `amount` from one integer key to another only if `from` has enough. Atomic. */
  transfer(from: string, to: string, amount: number): Promise<boolean>;
}

// Check and change in ONE step on the Redis server, so parallel requests can't both pass the check.
const TRANSFER_LUA = `
local bal = tonumber(redis.call('GET', KEYS[1]) or '0')
local amt = tonumber(ARGV[1])
if amt <= 0 or bal < amt then return 0 end
redis.call('DECRBY', KEYS[1], amt)
redis.call('INCRBY', KEYS[2], amt)
```

```
return 1`;

class UpstashStore implements Store {
  constructor(private url: string, private token: string) {}
  private async run(cmd: (string | number)[]): Promise<unknown> {
    const res = await fetch(this.url.replace(/\/$/, ""), {
      method: "POST",
      headers: { authorization: `Bearer ${this.token}`, "content-type": "application/json" },
      body: JSON.stringify(cmd),
      signal: AbortSignal.timeout(3_000),
      cache: "no-store",
    });
    const out = (await res.json()) as { result?: unknown; error?: string };
    if (!res.ok || out.error) throw new Error("store error");
    return out.result;
  }
  async count(key: string, windowMs: number) {
    await this.run(["SET", key, 0, "PX", windowMs, "NX"]); // creates the window once, with its expiry
    return Number(await this.run(["INCR", key])); // INCR keeps the expiry
  }
  async uncount(key: string) { await this.run(["DECR", key]); }
  async setNx(key: string, value: string, ttlMs?: number) {
    const ttl = ttlMs ? ["PX", Math.max(1, Math.round(ttlMs))] : [];
    return (await this.run(["SET", key, value, "NX", ...ttl])) === "OK";
  }
  async get(key: string) { return (await this.run(["GET", key])) as string | null; }
  async transfer(from: string, to: string, amount: number) {
    return Number(await this.run(["EVAL", TRANSFER_LUA, 2, from, to, amount])) === 1;
  }
}

// Single-threaded: no `await` between check and write, so each method is atomic within one process.
class MemoryStore implements Store {
  private kv = new Map<string, { v: string; exp: number }>();
  private live(k: string) { const e = this.kv.get(k); if (e && e.exp <= Date.now()) this.kv.delete(k);
return this.kv.get(k); }
  async count(k: string, windowMs: number) {
    const e = this.live(k);
    const n = e ? Number(e.v) + 1 : 1;
    this.kv.set(k, { v: String(n), exp: e?.exp ?? Date.now() + windowMs });
    return n;
  }
  async uncount(k: string) { const e = this.live(k); if (e) e.v = String(Number(e.v) - 1); }
  async setNx(k: string, v: string, ttlMs?: number) {
    if (this.live(k)) return false;
    this.kv.set(k, { v, exp: ttlMs ? Date.now() + ttlMs : Infinity });
    return true;
  }
  async get(k: string) { return this.live(k)?.v ?? null; }
  async transfer(from: string, to: string, amount: number) {
    const bal = Number(this.live(from)?.v ?? 0);
```

```
    if (amount <= 0 || bal < amount) return false;
    this.kv.set(from, { v: String(bal - amount), exp: Infinity });
    this.kv.set(to, { v: String(Number(this.live(to)?.v ?? 0) + amount), exp: Infinity });
    return true;
  }
}

// Every call fails with 503: production must never silently fall back to per-instance memory.
const unavailable: Store = new Proxy({} as Store, { get: () => async () => { throw new HttpError(503,
"Temporarily unavailable"); } });

let instance: Store | undefined;
export function store(): Store {
  if (instance) return instance;
  const url = process.env.UPSTASH_REDIS_REST_URL, token = process.env.UPSTASH_REDIS_REST_TOKEN;
  if (url && token) instance = new UpstashStore(url, token);
  else if (process.env.NODE_ENV !== "production" || process.env.ALLOW_MEMORY_STORE === "true") instance
= new MemoryStore();
  else instance = unavailable;
  return instance;
}
```

Use integer amounts only (cents, Wh), never floats. For a change that touches more than two keys, write one Lua script for that exact operation, with all checks inside it (R-BIZ-05).

`src/lib/limit.ts`: count first, then decide; trusted IP only

```
import "server-only";
import { store } from "./store";
import { HttpError } from "./http-error";

export async function requireRateLimit(key: string, max: number, windowMs: number) {
  let n: number;
  try { n = await store().count(`rl:${key}`, windowMs); } catch { throw new HttpError(503, "Temporarily
unavailable"); }
  if (n > max) throw new HttpError(429, "Too many requests");
}

// Only Vercel guarantees this header (it overwrites what the client sent). Elsewhere: unknown, so no IP
bucket.
export function clientIp(request: Request): string | null {
  if (!process.env.VERCEL) return null;
  return request.headers.get("x-forwarded-for")?.split(",")[0].trim() || null;
}

// Call BEFORE checking the password: every attempt is reserved atomically, so parallel guesses can't
slip through.
// Call the returned refund() after a CORRECT password, so only failures keep counting (no lockout of
the real user).
export async function requireLoginAttempt(account: string, request: Request): Promise<() =>
Promise<void>> {
```

```
  const window = 15 * 60_000;
  const keys: string[] = [];
  const reserve = async (key: string, max: number) => { await requireRateLimit(key, max, window);
keys.push(`rl:${key}`); };
  const ip = clientIp(request);
  if (ip) {
    await reserve(`login:${account}|${ip}`, 5); // strict: one attacker, one account
    await reserve(`login-ip:${ip}`, 50);
  }
  await reserve(`login-acct:${account}`, 100); // any IP: a ceiling against IP rotation, high enough to
be a slowdown
  return async () => { for (const k of keys) await store().uncount(k).catch(() => {}); };
}
```

`src/lib/csrf.ts`: Origin check for route handlers

```
import "server-only";
import { HttpError } from "./http-error";

// Same rule as Next.js uses for server actions: Origin must match the Host header.
// No env var needed, so it also works on Vercel preview URLs.
export function requireSameOrigin(request: Request) {
  const origin = request.headers.get("origin");
  if (!origin) {
    if (request.headers.get("sec-fetch-site") === "cross-site") throw new HttpError(403, "Forbidden");
    return;
  }
  let ok = false;
  const host = request.headers.get("host");
  try { ok = !!host && new URL(origin).host === host; } catch {}
  if (!ok) throw new HttpError(403, "Forbidden");
}
```

This compares against the host that was called, which is the same rule Next.js uses for server actions. Compare with the `Host` header and not `request.url`: under `next start`, `request.url` always says `localhost`, so `127.0.0.1` got 403.

Example route: `src/app/api/claims/[id]/vote/route.ts`

```
import { z } from "zod";
import { store } from "@/lib/store";
import { requireUser } from "@/lib/auth";
import { requireAccess } from "@/lib/access";
import { requireSameOrigin } from "@/lib/csrf";
import { requireRateLimit } from "@/lib/limit";
import { validatedJson } from "@/lib/validate";
import { HttpError, safeError } from "@/lib/http-error";
import { getClaim } from "@/lib/claims"; // your data-access function

const schema = z.object({ value: z.enum(["up", "down"]) }).strict();
```

```
export async function POST(request: Request, ctx: { params: Promise<{ id: string }> }) {
  try {
    requireSameOrigin(request);
    const user = await requireUser();                      // identity from the session only
    await requireRateLimit(`vote:${user.id}`, 60, 60_000);
    const { id } = await ctx.params;
    const claim = requireAccess(user, await getClaim(z.string().max(64).parse(id)));
    const { value } = await validatedJson(request, schema);
    const first = await store().setNx(`vote:${claim.id}:${user.id}`, value); // atomic: 1 vote
    if (!first) throw new HttpError(409, "Already voted");
    return Response.json({ ok: true }, { headers: { "Cache-Control": "no-store" } });
  } catch (e) { return safeError(e); }
}
```

`next.config.ts`: headers (tested in production apps)

```
import type { NextConfig } from "next";

const securityHeaders = [
  { key: "X-Frame-Options", value: "DENY" },
  { key: "Content-Security-Policy", value: "frame-ancestors 'none'; base-uri 'self'; form-action 'self';
object-src 'none'" },
  { key: "X-Content-Type-Options", value: "nosniff" },
  { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
  { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=()" },
  { key: "Strict-Transport-Security", value: "max-age=63072000; includeSubDomains" },
];

const nextConfig: NextConfig = {
  poweredByHeader: false,
  async headers() {
    return [
      { source: "/:path*", headers: securityHeaders },
      // Later entries win. Downloads and public share links get a stricter CSP (R-HTTP-01).
      { source: "/api/files/:path*", headers: [{ key: "Content-Security-Policy", value: "default-src
'none'; sandbox" }] },
      { source: "/s/:path*", headers: [{ key: "Content-Security-Policy", value: "default-src 'none';
sandbox" }, { key: "Referrer-Policy", value: "no-referrer" }, { key: "X-Robots-Tag", value: "noindex" }]
},
    ];
  },
};

export default nextConfig;
```

Merge this into the scaffold's `next.config.ts` (keep its existing settings). Adjust the two paths to your download and share routes.

CSP choice: this CSP blocks clickjacking, `<base>` hijacking and plugins, and doesn't break Next.js. Don't add `script-src 'self' 'unsafe-inline'`: `'unsafe-inline'` may *itself* be flagged. If there's time, the strongest option is a nonce-based CSP. It requires dynamic rendering, so test it in production before submitting.

`.gitignore` and `.env.example`

```
node_modules/
.next/
coverage/
*.log
.env*
!.env.example
uploads/
*.pem
*.key
*.local.md
```

```
# placeholders only
SESSION_SECRET=generate-with-openssl-rand-base64-48
AUTH_USERS=id:salt:scrypthash,id:salt:scrypthash   # from scripts/create-users.mjs, never plain
passwords
GUEST_ACCESS=on
# Only for a single-instance demo without Upstash; listed as a trade-off in the README.
ALLOW_MEMORY_STORE=false
UPSTASH_REDIS_REST_URL=https://example.upstash.io
UPSTASH_REDIS_REST_TOKEN=replace-me
ANTHROPIC_API_KEY=replace-me
```

Section 6: README security template (step 5)

Ignore an accepted risk **per finding** in the Aikido UI, and also note it in the README. Don't invent inline suppression comments for the AI Code Audit.

```
## Security

| Control | Where | Test |
|---|---|---|
| Signed session cookie (HMAC, HttpOnly, SameSite=Strict, 8h), revoked at logout | `src/lib/auth.ts` |
`auth.test.ts` |
| Every route checks session + owner/tenant (404 on mismatch) | `src/lib/access.ts`, each `route.ts` |
A/B test |
| Origin check on mutations | `src/lib/csrf.ts` | |
| Input validation (zod, 16 KB limit) | `src/lib/validate.ts` | |
| Rate limits (login, LLM), Upstash, fail closed | `src/lib/limit.ts` | |
| RAG filtered by access before retrieval | `src/lib/<file>.ts` | |

**Known trade-offs (deliberate):**
- <What we're keeping>: <why it's acceptable here (synthetic data, prototype)> → <what production would
do>. Aikido finding: <link/ID>.
```

Example: "*One person can create several accounts and vote more than once. Sign-ups are limited to 5 per IP per hour; single sign-on would remove this.*"

Section 7: Common agent mistakes

Trap	Problem	Do this instead
"`/dashboard` is protected in `proxy.ts`"	API routes and server actions are reachable directly	`requireUser()` + permission check in every entry point
Auth check only on the page	The server actions on that page stay open	Check again inside every action
`const { userId } = await req.json()`	IDOR, impersonation	`user.id` from the session
`redis.get('project:' + id)` without an owner check	Data from other tenants leaks	`requireAccess(user, obj)`
`setCookie('user', JSON.stringify(user))` / base64	Anyone can forge it	A signed or opaque session
`localStorage.setItem('token', …)`	Stolen through XSS	An HttpOnly cookie
`catch { return demoUser }`	Fail-open auth	503
`get(voted)` and then `incr(votes)`	Duplicate votes in a race	`SET NX` / a transaction
`update({ data: body })`	Mass assignment	A `.strict()` schema with allowed fields
`fetch(body.url)`	SSRF	Fixed URLs
`dangerouslySetInnerHTML={{__html: answer}}` / `rehype-raw`	XSS through the LLM or documents	Plain text, or sanitized Markdown
`NEXT_PUBLIC_OPENAI_API_KEY` to "fix" a build error	Key in the browser bundle	A server route that calls the provider
`new Map()` as a rate limiter on Vercel	Each instance has its own counter	Upstash
No Origin check "because SameSite" on a route handler	CSRF	`requireSameOrigin()`
`Access-Control-Allow-Origin: *` with cookies	Cross-origin data leak	No CORS, or an exact origin
RAG searches everything, then asks the LLM to filter	The model sees forbidden data	Filter before retrieval
A general DB, shell or fetch tool for the model	Excessive agency	Small typed tools with a permission check
`err.message` or a stack trace in the response	Information leak	`safeError()`
`console.log(req.headers)`	Cookies and tokens in the logs	Log only the event and outcome
A broad `.aikido` exclude to "clear" findings	Real bugs hidden, and it looks bad	Fix, or ignore one finding with a reason
`peek` the login counter, check the password, then count the failure	Parallel guesses all pass the peek	`INCR` first, then decide (`requireLoginAttempt`)

Trap	Problem	Do this instead
`x-forwarded-for` as the IP everywhere, `"unknown"` as the fallback	Unlimited guesses by rotating the header; one shared lockout bucket	`clientIp()` trusts it only on Vercel, plus a per-account limit
A `SET NX PX` lock around several balance updates	The lock expires mid-transaction: double spend	One Lua script or compare-and-set on one document
A rule checked only when a request is created	Two pending requests together break it at approval	Re-check at every step that commits it
EUR 50 or votes for every new sign-up	Fake accounts mint money and stuff votes	Free credit can only be spent; votes only from seeded or verified accounts
`displayName = email.split("@")[0]`	Personal data on public pages	A chosen display name or a pseudonym
Sources and question in one string with `QUESTION:` markers	A document takes over the question and triggers tools	Separate content blocks; the code decides which tools may run
Download CSP set in the route handler	The global `next.config` headers overwrite it	A later, more specific `headers()` entry
Comparing Origin with `request.url`	The app's own UI gets 403 on `127.0.0.1`	Compare with the `Host` header
A rate limiter that returns "allowed" when the store is down	Abuse limits disappear exactly when the store struggles	`catch` → 503 (fail closed)
No `import "server-only"` in files that read secrets	A client import can pull secrets into the browser bundle	`import "server-only"` at the top of every server module
Production without Upstash quietly uses memory	Per-instance state: logout, limits and balances break	503 unless `ALLOW_MEMORY_STORE=true`
`throw` at import when an env var is missing	`next build` crashes	Check on use, return 503
Demo passwords in `README.md`	Secrets finding, a free login for anyone	Submission form, guest mode, or a harmless documented account
A "production-ready" Dockerfile or Kubernetes YAML nobody asked for	Dozens of IaC findings (root user, `latest` tag, writable filesystem)	No infra files unless the case needs them
`if (password === stored)` or `token === expected`	"Timing attack" (High)	`timingSafeEqual`
`jwt.decode(token)` to read the user	Forged tokens accepted	`jwt.verify` with a fixed algorithm, or the section 5 session
`<script src="https://cdn.polyfill.io/...">`	"Malicious domain" (Critical)	npm packages, or a pinned CDN URL with `integrity=`
Pushing straight to an unprotected `main` until the end	GitHub findings `github_006/002/004`	Branch protection in the last 20 minutes
"Make Aikido pass" as the only instruction	Hiding instead of fixing	This document